

UsLab-Eng-01 Master Plan

RKE2 + TopoLVM + MetalLB + Nginx Proxy Manager — UsLab-Eng-01 Master Build Plan

Cluster: `UsLab-Eng-01` **Distribution:** RKE2 v1.33.x (Calico CNI) **Storage:** TopoLVM — **thick** node-local LVM on NVMe, **one VG + StorageClass per node** (Lumens-style) **Edge (initial stage):** **MetalLB** (L2) + **Nginx Proxy Manager** (reverse proxy + Let's Encrypt). **No ingress controller. Async replication (optional):** VolSync, `copyMethod: Direct` (thick has no snapshots) **Topology:** 1 master + invoker (`roso`) · 4 app/storage workers · 1 invoker worker (`poco`)

Why no ingress yet? The initial stage keeps the edge simple: MetalLB hands a LAN IP to Nginx Proxy Manager (NPM), which proxies `*.antbrains.com` and manages TLS in a UI. The Traefik v3 + cert-manager ingress design is **deferred to Phase 2** (a few months out) and parked under `Traefik.phase2/`.

No Ceph. This cluster is pure node-local NVMe via TopoLVM — the move off Ceph is to kill the network-hop latency. See `storage-performance.html`.

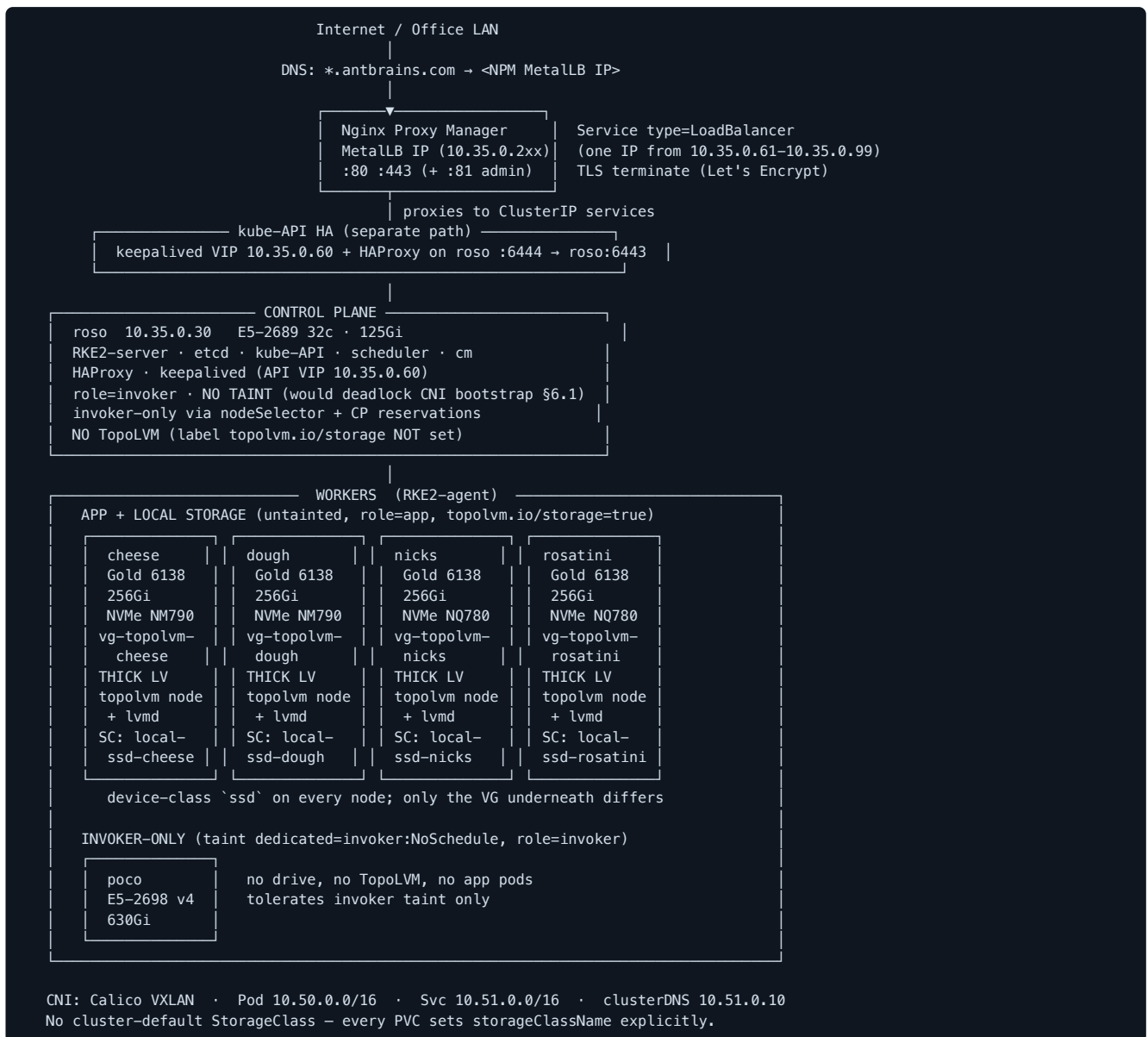
0. Parameters to confirm before install

Item	Value	Note
Node subnet	<code>10.35.0.0/24</code>	matches old uslab cluster
API VIP	<code>10.35.0.60</code>	kube-API endpoint / TLS-SAN (keepalived, optional with single master)
MetalLB pool	<code>10.35.0.61-10.35.0.99</code>	LoadBalancer IPs for apps; NPM took <code>10.35.0.61</code> . Above the VIP, outside DHCP, no overlap with node IPs
Master IP (roso)	<code>10.35.0.30</code>	master + invoker worker
Worker IPs	<code>cheese 10.35.0.51 · dough 10.35.0.52 · rosatini 10.35.0.53 · nicks 10.35.0.54 · poco <IP_poco></code>	static, in <code>10.35.0.x</code> (see §1.5)
Cluster name	<code>uslab-eng-01</code>	
Domain	<code>antbrains.com</code>	external DNS zone
Pod CIDR	<code>10.50.0.0/16</code>	non-overlapping with old uslab (10.48/49) and punelab (172.30/31)
Service CIDR	<code>10.51.0.0/16</code>	cluster DNS <code>10.51.0.10</code>
RKE2 version	<code>v1.33.1+rke2r1</code>	pin a current v1.33 patch before install
TopoLVM chart	<code>topolvm/topolvm v15.x</code>	latest stable
MetalLB chart	<code>metallb/metallb latest</code>	L2 mode
NPM image	<code>jc21/nginx-proxy-manager</code>	pin a tag/digest for production
VolSync chart	<code>backube/volsync latest</code>	optional; Direct mode
cert-manager chart	<code>jetstack/cert-manager v1.18</code>	required by the TopoLVM webhook

Hostnames are lowercase RFC1123: `roso` , `cheese` , `dough` , `nicks` , `rosatini` , `poco` .

1. Architecture

1.1 Logical topology



1.2 Request flow — external HTTPS (initial stage)

```
user-agent
| HTTPS https://app.antbrains.com
▼ DNS *.antbrains.com → NPM's MetalLB IP
▼ MetalLB L2 (one node ARPs the VIP) → Service npm-proxy (LoadBalancer)
▼ Nginx Proxy Manager pod – TLS terminate (Let's Encrypt cert in NPM)
▼ proxy_pass → ClusterIP service (svc.ns.svc.cluster.local)
▼ Pod
```

1.3 Storage flow — PVC (thick, per node)

```
workload pod (PVC: storageClassName = local-ssd-<node>, explicit)
▼ WaitForFirstConsumer – bind waits until the pod schedules
▼ allowedTopologies pins the PVC to that ONE node
▼ topolvm-controller creates a LogicalVolume CR
▼ lvmd on that node carves a THICK linear LV from vg-topolvm-<node>
▼ topolvm-node mounts the LV (xfs) into the pod
▼ pod gets raw NVMe speed; the pod is now pinned to that node
```

1.4 Async replication flow — VolSync (Direct, optional)

```
source-ns/data (PVC, local-ssd-cheese)
| scheduled trigger (e.g. hourly)
▼ mover co-mounts the LIVE PVC (same node, RW0) - NO snapshot (thick)
▼ rsync-over-TLS push → target-ns/data-dest (PVC, local-ssd-dough)
For consistent DB copies: quiesce the writer, or replicate an app-level dump.
RPO = schedule interval.
```

1.5 Node inventory

Node	IP	Role	OS	CPU	RAM	Root SSD	NVMe (LVM PV)	LVM VG	k8s labels	k8s taints
roso	10.35.0.30	master + invoker	Ubuntu 24.04.1	Xeon E5-2689 32c	125Gi	—	—	—	role=invoker	none (untainted — see §6.1)
cheese	10.35.0.51	app + storage	24.04.4	Xeon Gold 6138 80c	256Gi	931.5G BX500	3.7TiB Lexar NM790	vg-topolvm-cheese	role=app, topolvm.io/storage=true	—
dough	10.35.0.52	app + storage	24.04.4	Xeon Gold 6138 80c	256Gi	931.5G BX500	3.7TiB Lexar NM790	vg-topolvm-dough	role=app, topolvm.io/storage=true	—
rosatini	10.35.0.53	app + storage	24.04.4	Xeon Gold 6138 80c	256Gi	931.5G BX500	3.7TiB Lexar NQ780	vg-topolvm-rosatini	role=app, topolvm.io/storage=true	—
nicks	10.35.0.54	app + storage	24.04.4	Xeon Gold 6138 80c	256Gi	931.5G BX500	3.7TiB Lexar NQ780	vg-topolvm-nicks	role=app, topolvm.io/storage=true	—
poco	<IP_poco>	invoker	24.04.1	Xeon E5-2698 v4 80c	630Gi	—	—	—	role=invoker	dedicated=invoker:NoSchedule

Role split

- App + storage: `cheese`, `dough`, `nicks`, `rosatini` (untainted, `topolvm.io/storage=true`).
- Invoker workloads land on `roso` (master, also a worker) and `poco` (no storage label → no TopoLVM there).
- `roso` is UNTainted; `poco` keeps the `dedicated=invoker:NoSchedule` taint. This split is deliberate — see §6.1.

`roso` is master AND worker. It runs the full control plane (etcd, kube-API, scheduler, controller-manager) and schedules invoker pods. It is not tainted: RKE2's core add-on installer Jobs (Calico, CoreDNS, ...) are pinned to the control-plane node, and a taint there deadlocks the whole CNI bootstrap (we hit this — §6.1). Instead, app pods are kept off `roso` via `nodeSelector` (they target `role=app` / per-node storage classes), and the control plane is protected by fixed CPU/RAM reservations (`system-reserved` / `kube-reserved`, §6.1). `poco` is a pure invoker worker and safely keeps the taint.

poco IP not yet provided — fill `<IP_poco>` before install.

2. Network

Component	Value
Node subnet	10.35.0.0/24
API VIP (keepalived)	10.35.0.60
MetalLB pool	10.35.0.61-10.35.0.99
Pod CIDR	10.50.0.0/16
Service CIDR	10.51.0.0/16
Cluster DNS	10.51.0.10
External domain	antbrains.com
DNS records	*.antbrains.com → NPM's MetalLB IP; k8s-api → 10.35.0.60

Ports to open (firewall):

Port	Proto	Where	Purpose
22	TCP	all	SSH
6443	TCP	roso	kube API
6444	TCP	roso	HAProxy API frontend
9345	TCP	roso	RKE2 supervisor (node join)
2379-2380	TCP	roso	etcd
10250	TCP	all	kubelet
80, 443	TCP	app nodes	NPM (MetalLB L2 targets the app nodes)
81	TCP	internal only	NPM admin UI — do NOT expose publicly
30000-32767	TCP	all	NodePort range
8472	UDP	all	Calico VXLAN
7946	TCP+UDP	all	MetalLB speaker memberlist

VolSync rsync-over-TLS stays inside the cluster (ClusterIP) — no external ports.

3. Storage Design — TopoLVM, thick, one VG + StorageClass per node

TopoLVM gives each PVC a Logical Volume on the **node where the pod runs** — raw NVMe speed, no network hops, no Ceph daemons. This cluster uses the **Lumens pattern**: **thick** LVM (no thin pools, no CSI snapshots), a **per-node VG** named after the node, and a **per-node StorageClass**.

Node	Drive	VG	Device class	StorageClass
cheese	Lexar NM790 4TB	vg-topolvm-cheese	ssd	local-ssd-cheese
dough	Lexar NM790 4TB	vg-topolvm-dough	ssd	local-ssd-dough
nicks	Lexar NQ780 4TB	vg-topolvm-nicks	ssd	local-ssd-nicks
rosatini	Lexar NQ780 4TB	vg-topolvm-rosatini	ssd	local-ssd-rosatini

Why thick (and what it costs). Thick reserves each PVC's space up front and maps straight to physical extents (`dm-linear`) — the lowest, flattest latency and nothing to monitor (a full VG just makes new PVCs Pending; it never blocks live writes). The trade-off is **no CSI snapshots**, so VolSync runs in Direct mode (§8.6). This matches the Lumens cluster exactly. (Thin pools would buy snapshots + overprovisioning at the cost of pool-fill risk and monitoring — see [thin-vs-thick-implementation.html](#).)

Constraints of local storage

- **RWO, node-local.** A PVC binds to one node; the pod is pinned there.
- **No default StorageClass.** Every PVC MUST set `storageClassName` or it stays Pending.
- **Node down → its PVCs unreachable** (data intact, just unreachable). VolSync/app backups cover DR.
- **Per-node capacity = that one drive** (~3.7 TiB). `vgextend` to grow.
- **reclaimPolicy: Retain** — deleting a PVC leaves the PV + LV (data not erased).

4. Server Preparation (ALL 6 nodes)

```
# packages – lvm2 is critical for TopoLVM (no thin-provisioning-tools needed: thick)
sudo apt update && sudo apt upgrade -y
sudo apt install -y curl wget git vim htop net-tools \
  software-properties-common apt-transport-https ca-certificates gnupg \
  lsb-release nfs-common open-iscsi lvm2 cryptsetup chrony xfsprogs

# time sync (etcd-critical)
sudo systemctl enable --now chronyd
timedatectl status | grep synchronized      # must read: yes

# hostname – LOWERCASE, run per node
sudo hostnamectl set-hostname <THIS_NODE>   # roso | cheese | dough | nicks | rosatini | poco

# /etc/hosts (all nodes)
cat << 'EOF' | sudo tee -a /etc/hosts
10.35.0.30    roso
10.35.0.51    cheese
10.35.0.52    dough
10.35.0.53    rosatini
10.35.0.54    nicks
<IP_poco>    poco
10.35.0.60    k8s-api.antbrains.com
EOF

# swap off
sudo swapoff -a
sudo sed -i 's/ swap / s/^/#/' /etc/fstab

# kernel modules (THICK: no dm_thin_pool needed)
cat << 'EOF' | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
ip_vs
ip_vs_rr
ip_vs_wrr
ip_vs_sh
nf_conntrack
EOF
sudo modprobe overlay br_netfilter ip_vs ip_vs_rr ip_vs_wrr ip_vs_sh nf_conntrack

# sysctl
cat << 'EOF' | sudo tee /etc/sysctl.d/99-kubernetes.conf
net.ipv4.ip_forward = 1
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.netfilter.nf_conntrack_max = 1000000
fs.inotify.max_user_instances = 8192
fs.inotify.max_user_watches = 524288
fs.file-max = 2097152
vm.max_map_count = 524288
net.core.somaxconn = 32768
EOF
sudo sysctl --system

# ulimits + containerd limits
cat << 'EOF' | sudo tee /etc/security/limits.d/kubernetes.conf
* soft nofile 1048576
* hard nofile 1048576
* soft nproc 1048576
* hard nproc 1048576
* soft memlock unlimited
* hard memlock unlimited
EOF
sudo mkdir -p /etc/systemd/system/containerd.service.d
cat << 'EOF' | sudo tee /etc/systemd/system/containerd.service.d/limits.conf
[Service]
LimitNOFILE=1048576
LimitNPROC=1048576
LimitMEMLOCK=infinity
EOF
sudo systemctl daemon-reload
```

4.1 NVMe wipe + LVM setup — cheese, dough, nicks, rosatini ONLY

Never touch the 931.5G Crucial root SSD. Confirm the NVMe first. THICK = PV + VG only (no thin pool). The VG is **named after the node**.

⚠️ These boxes ship with the NVMe already carrying an old OS layout — an EFI + boot + LVM (`ubuntu-vg`) partition set — even though the running root is on the SATA SSD (`sda` , VG `ubuntu-vg-1`). LVM holds that disk **busy**, so a plain `wipefs` / `pvccreate` fails with `Device or`

resource busy / device has a signature. You must tear the old LVM + partitions down first.

```
DEV=/dev/nvme0n1
lsblk -dno NAME,SIZE,MODEL $DEV          # confirm it is the 3.7T Lexar, NOT root

# --- SAFETY: confirm the running root is on sda, and which VG sits on the NVMe ---
findmnt -no SOURCE /                     # must be ../ubuntu--vg--l... on sda3 (NOT nvme)
sudo pvs                                 # note the VG on /dev/nvme0n1p3 (e.g. "ubuntu-vg")
lsblk "$DEV"                             # see the existing partitions/LVM

# --- Tear down the OLD LVM on the NVMe (the VG that is NOT the root VG) ---
OLDVG=ubuntu-vg                         # <-- the NVMe VG from `pvs`, NOT ubuntu-vg-l (root)
sudo vgchange -an "$OLDVG"               # deactivate (releases the busy device)
sudo vgremove -f "$OLDVG"
sudo pvremove -ff "${DEV}p3"

# --- Wipe partitions + clear the kernel's stale table ---
sudo wipefs -a "${DEV}p1" "${DEV}p2" "${DEV}p3" 2>/dev/null
sudo sgdisk --zap-all "$DEV"
sudo wipefs -a "$DEV"
sudo partprobe "$DEV" || sudo reboot     # if still busy, reboot then continue
lsblk "$DEV"                             # should now show NO partitions

# --- Create the thick per-node VG ---
sudo blkdiscard "$DEV"                   # optional fast TRIM
sudo pvcreate "$DEV"
sudo vgcreate vg-topolvm-cheese "$DEV"   # per node:
#   dough    -> vg-topolvm-dough
#   nicks    -> vg-topolvm-nicks
#   rosatini -> vg-topolvm-rosatini
sudo pvs && sudo vgs                     # confirm VG size ≈ 3.73 TiB, all free
```

Skip this whole step on **roso** and **poco**. If a node's NVMe is already blank, the teardown block is a no-op — go straight to **pvcreate** / **vgcreate**.

5. Load Balancer for kube-API (HAProxy + keepalived on **roso**)

This is the **control-plane HA path only** — application traffic goes through MetalLB + NPM (§9), not this. (Single master today, so this is a thin shim that also gives a stable API endpoint and room to add control-plane nodes later.)

/etc/haproxy/haproxy.cfg:

```
global
    log stdout local0
    maxconn 4000
defaults
    mode tcp
    log global
    option tcplog
    timeout connect 10s
    timeout client 1m
    timeout server 1m
    retries 3

frontend k8s-api-frontend
    bind *:6444
    default_backend k8s-api-backend
backend k8s-api-backend
    option tcp-check
    balance roundrobin
    server roso 10.35.0.30:6443 check fall 3 rise 2

listen stats
    bind *:8404
    mode http
    stats enable
    stats uri /stats
    stats refresh 30s
    stats auth admin:<STATS_PW>
```

/etc/keepalived/keepalived.conf:

```

vrp_script chk_haproxy {
    script "/bin/curl -f http://localhost:8404/stats || exit 1"
    interval 2
    weight -2
    fall 3
    rise 2
}
vrp_instance VI_1 {
    state MASTER
    interface <IFACE>          # ip -br addr | grep 10.35.0
    virtual_router_id 61       # unique on this L2 segment; MUST differ from MetalLB
    priority 110
    advert_int 1
    authentication { auth_type PASS; auth_pass <VRP_PW> }
    virtual_ipaddress { 10.35.0.60/24 }
    track_script { chk_haproxy }
}

```

```

sudo apt install -y haproxy keepalived
sudo haproxy -c -f /etc/haproxy/haproxy.cfg
sudo keepalived -t -f /etc/keepalived/keepalived.conf
sudo systemctl enable --now haproxy keepalived
ip -br addr | grep 10.35.0.60      # VIP must be on roso

```

Keep keepalived (VRRP) and MetalLB L2 separate. keepalived owns `10.35.0.60` (API). MetalLB owns `10.35.0.61-10.35.0.99` (apps). Different IPs, and the keepalived `virtual_router_id` must not collide with anything else on the segment.

6. Control Plane — RKE2 Server on `roso`

`roso` is the master AND hosts invoker pods (tainted `dedicated=invoker`). RKE2 disables `rke2-canal` (using Calico), and both bundled ingresses — we use **no ingress controller** in the initial stage.

```

sudo mkdir -p /etc/rancher/rke2
cat << 'EOF' | sudo tee /etc/rancher/rke2/config.yaml
tls-san:
  - k8s-api.antbrains.com
  - 10.35.0.60
  - 10.35.0.30
  - roso

cluster-name: "uslab-eng-01"
cluster-cidr: 10.50.0.0/16
service-cidr: 10.51.0.0/16
cluster-dns: 10.51.0.10
cluster-domain: cluster.local

cluster-init: true
write-kubeconfig-mode: "0644"

disable:
  - rke2-canal
  - rke2-ingress-nginx
  - rke2-traefik
cni:
  - calico

node-ip: 10.35.0.30
advertise-address: 10.35.0.30
etcd-expose-metrics: true

node-name: roso
# NO node-taint on roso. A taint here blocks RKE2's core add-on installer Jobs
# (rke2-calico, rke2-coredns, ...) which are pinned to the control-plane node, and
# the whole CNI bootstrap deadlocks (FailedScheduling: untolerated taint). We hit
# exactly this. App pods are kept off roso via nodeSelector instead (§6.1, §10).
node-label:
  - "node.antbrains.com/role=invoker"

# Reservations so the control plane (etcd/apiserver/scheduler/cm) stays healthy
# while roso ALSO runs invoker pods. See §6.1 for the rationale.
# system-reserved -> OS, sshd, containerd, kubelet host daemons
# kube-reserved -> headroom for the RKE2 control-plane pods + kubelet
# eviction-hard -> kubelet starts evicting before the box is truly out
# 32c / 125Gi: ~6 vCPU + ~16Gi held back; ~26 vCPU / ~105Gi left for invoker pods.
kubelet-arg:
  - "max-pods=150"
  - "system-reserved=cpu=2,memory=4Gi"
  - "kube-reserved=cpu=4,memory=12Gi"
  - "eviction-hard=memory.available<8%,nodefs.available<10%,imagefs.available<10%"
  - "kube-reserved-cgroup=/podruntime.slice"
  - "system-reserved-cgroup=/system.slice"
EOF

curl -sfl https://get.rke2.io | sudo INSTALL_RKE2_VERSION="v1.33.1+rke2r1" sh -
sudo systemctl enable --now rke2-server.service
sudo journalctl -u rke2-server -f # wait until ready

# kubectl
sudo cp /var/lib/rancher/rke2/bin/kubectl /usr/local/bin/
mkdir -p ~/.kube && sudo cp /etc/rancher/rke2/rke2.yaml ~/.kube/config
sudo chown $(id -u):$(id -g) ~/.kube/config
echo 'export PATH=$PATH:/var/lib/rancher/rke2/bin' >> ~/.bashrc
kubectl get nodes

sudo cat /var/lib/rancher/rke2/server/node-token # for joining workers

```

6.1 roso reservation strategy (master + invoker worker)

roso wears two hats: it is the **only control-plane node** (etcd, kube-API, scheduler, controller-manager) **and** an invoker worker. Without guardrails a burst of invoker pods could starve etcd/apiserver and take the whole cluster down.

⚠ **DO NOT taint roso**. We originally tainted it `dedicated=invoker:NoSchedule` and the cluster never came up: RKE2's core add-on installer Jobs (`helm-install-rke2-calico`, `-coredns`, ...) are **pinned to the control-plane node** via node affinity. The custom taint on the only control-plane node left them **Pending** (`FailedScheduling: untolerated taint {dedicated: invoker}`), so the CNI never installed and every worker stayed **NotReady**. Removing the taint let the bootstrap complete. A taint on the sole control-plane node fights RKE2 — use the nodeSelector approach below instead. (`poco` has no control plane, so it safely keeps the taint.)

Three mechanisms keep the control plane safe on the untainted **roso**:

1. **nodeSelector** instead of a taint. App pods target `role=app` (and their per-node TopoLVM StorageClasses only bind on storage nodes — `roso` has none), so app workloads don't land on `roso`. Invoker pods target `role=invoker` (§10).
2. **system-reserved** + **kube-reserved** carve fixed CPU/RAM out of the node's allocatable pool for the OS and the control plane, so pods can never consume the whole box:

Bucket	CPU	Memory	Protects
<code>system-reserved</code>	2	4Gi	OS, sshd, containerd, kubelet
<code>kube-reserved</code>	4	12Gi	RKE2 control-plane pods + kubelet headroom
<code>eviction-hard</code>	—	8%	kubelet evicts before true OOM
Left for invoker pods (Allocatable)	~26	~105Gi	of 32c / 125Gi

3. **Priority**. The control-plane pods run at `system-node-critical` / `system-cluster-critical` (RKE2 default), so under pressure the scheduler **preempts invoker pods** to keep etcd/apiserver alive. Therefore give invoker workloads a **normal (default) PriorityClass** — do NOT mark them system-critical, or they'd compete with the control plane.

Tune the numbers to real load: watch `kubectl top node roso` and `kubectl -n kube-system top pod`. If etcd ever shows disk/CPU pressure, raise `kube-reserved` (and put heavy invoker load on `poco`). The single master has no HA — protecting it is the priority. If you ever add a 2nd/3rd control-plane node, you *can* taint roso then (add-on Jobs will have another CP node to run on).

7. Workers + cert-manager

7.1 App + storage workers — cheese, dough, nicks, rosatini

```
sudo mkdir -p /etc/rancher/rke2
cat << 'EOF' | sudo tee /etc/rancher/rke2/config.yaml
server: https://10.35.0.30:9345
token: <NODE_TOKEN>
node-name: <THIS_NODE>           # cheese | dough | nicks | rosatini
node-ip: <THIS_NODE_IP>          # cheese 10.35.0.51 · dough 10.35.0.52 · rosatini 10.35.0.53 · nicks 10.35.0.54
kubelet-arg:
- "max-pods=250"
- "system-reserved=cpu=4,memory=16Gi"
- "kube-reserved=cpu=2,memory=8Gi"
- "eviction-hard=memory.available<5%,nodefs.available<10%"
node-label:
- "node.antbrains.com/role=app"
EOF
curl -sfl https://get.rke2.io | sudo INSTALL_RKE2_VERSION="v1.33.1+rke2r1" INSTALL_RKE2_TYPE="agent" sh -
sudo systemctl enable --now rke2-agent.service
sudo journalctl -u rke2-agent -f
```

7.2 Invoker worker — poco

```
sudo mkdir -p /etc/rancher/rke2
cat << 'EOF' | sudo tee /etc/rancher/rke2/config.yaml
server: https://10.35.0.30:9345
token: <NODE_TOKEN>
node-name: poco
node-ip: <IP_poco>
node-taint:
- "dedicated=invoker:NoSchedule"
node-label:
- "node.antbrains.com/role=invoker"
kubelet-arg:
- "max-pods=200"
- "system-reserved=cpu=4,memory=16Gi"
- "kube-reserved=cpu=2,memory=8Gi"
- "eviction-hard=memory.available<5%,nodefs.available<10%"
EOF
curl -sfl https://get.rke2.io | sudo INSTALL_RKE2_VERSION="v1.33.1+rke2r1" INSTALL_RKE2_TYPE="agent" sh -
sudo systemctl enable --now rke2-agent.service
sudo journalctl -u rke2-agent -f
```

7.3 Verify

```
kubectl get nodes -o wide           # 6 Ready
kubectl describe node roso | grep -A2 Taints # dedicated=invoker:NoSchedule
kubectl describe node poco | grep -A2 Taints # dedicated=invoker:NoSchedule
kubectl get nodes -L node.antbrains.com/role
```

7.4 cert-manager (required by the TopoLVM webhook)

```
helm repo add jetstack https://charts.jetstack.io && helm repo update
kubectl create namespace cert-manager
helm install cert-manager jetstack/cert-manager -n cert-manager \
  --version v1.18.0 --set crds.enabled=true
kubectl -n cert-manager rollout status deploy/cert-manager
kubectl -n cert-manager rollout status deploy/cert-manager-webhook
```

8. Storage — TopoLVM (thick, per node) + VolSync

8.1 Label storage nodes

```
for n in cheese dough nicks rosatini; do
  kubectl label node $n topolvm.io/storage=true --overwrite
done
kubectl get nodes -L topolvm.io/storage # roso, poco must NOT be set
```

8.2 Install TopoLVM

Values live at [TopoLVM/topolvm-values.yaml](#) (one lvmd per node via [kubernetes.io/hostname](#), each pointing at its own `vg-topolvm-<node>`, device class `ssd`, thick, `storageClasses: []`).

⚠ **webhook.podMutatingWebhook.enabled: false is REQUIRED here** (set in the values file). We use capacity-aware scheduling via **CSIStorageCapacity** (`storageCapacityTracking.enabled: true`) with the **stock scheduler — no topolvm-scheduler extender**. The pod mutating webhook stamps a `topolvm.io/capacity` extended-resource request onto every PVC-mounting pod that *only* the (not-deployed) extender could satisfy → pods stick at `0/N nodes available: Insufficient topolvm.io/capacity` forever. We hit exactly this with NPM. Leaving the pod webhook **off** is what makes scheduling work.

```
helm repo add topolvm https://topolvm.github.io/topolvm && helm repo update
kubectl create namespace topolvm-system
kubectl label ns topolvm-system topolvm.io/webhook=ignore --overwrite
kubectl label ns kube-system topolvm.io/webhook=ignore --overwrite
helm upgrade --install topolvm topolvm/topolvm -n topolvm-system \
  -f TopoLVM/topolvm-values.yaml
kubectl -n topolvm-system rollout status deploy/topolvm-controller
```

8.3 StorageClasses (per node, no default, Retain)

```
kubectl apply -f TopoLVM/storageclasses.yaml
kubectl get sc # 4 local-ssd-* classes, NO default
kubectl get nodes -L topology.topolvm.io/node
```

8.4 Verify + smoke test

```
kubectl -n topolvm-system get pod -o wide # lvmd + node on the 4 storage nodes only
kubectl get csidrivers | grep topolvm.io
kubectl get csistoragecapacities -A -o wide

kubectl create ns demo
kubectl apply -f TopoLVM/pvc.yaml # local-ssd-cheese; online resize works
kubectl -n demo get pvc,pod -o wide
kubectl delete ns demo
```

8.5 Install VolSync (optional, Direct mode)

```
helm repo add backube https://backube.github.io/helm-charts && helm repo update
helm install volsync backube/volsync -n volsync-system --create-namespace
kubectl -n volsync-system rollout status deploy/volsync
```

Templates in [VolSync/](#) use `copyMethod: Direct` (thick has no snapshots). Put the destination on a **different node's** class than the source. For consistent database copies, quiesce the writer or replicate an app-level dump — see [VolSync/README.md](#).

9. Edge (initial stage) — MetalLB + Nginx Proxy Manager

No ingress controller. MetalLB hands a LAN IP to NPM, which proxies and does TLS.

9.1 MetalLB (L2)

```
helm repo add metallb https://metallb.github.io/metallb && helm repo update
kubectl create namespace metallb-system
helm install metallb metallb/metallb -n metallb-system
kubectl -n metallb-system rollout status deploy/metallb-controller

# edit 10.35.0.61-10.35.0.99 first, then:
kubectl apply -f /Users/kiranmane/Documents/Repository/krista_private_cloud/UsLab-Eng-01/MetalLB/metallb-pools.yaml
kubectl -n metallb-system get ipaddresspool,l2advertisement
```

9.2 Nginx Proxy Manager

```
kubectl apply -f /Users/kiranmane/Documents/Repository/krista_private_cloud/UsLab-Eng-01/NginxProxyManager/npm.yaml
kubectl -n npm get pvc,pod,svc -o wide
kubectl -n npm get svc npm-proxy -o jsonpath='{.status.loadBalancer.ingress[0].ip}'
# ^ point *.antbrains.com DNS at this IP
```

Admin UI (port 81) stays ClusterIP — reach it with `kubectl -n npm port-forward svc/npm-admin 8181:81`. First login `admin@example.com / changeme` → **change immediately**. Add a Proxy Host per app (forward to `svc.ns.svc.cluster.local`), request a Let's Encrypt cert in the UI. Details in [NginxProxyManager/README.md](#).

9.3 Verify

```
kubectl get svc -A | grep LoadBalancer      # npm-proxy has an EXTERNAL-IP
curl -kI https://app.antbrains.com           # after DNS + a proxy host + cert
```

10. Scheduling Invoker Workloads (roso + poco)

Invoker pods select the invoker nodes via `nodeSelector` and tolerate `poco`'s taint. App pods (no `role=invoker` selector) target `role=app` / per-node storage classes, so they don't land on `roso` / `poco`. Invoker workloads should not use TopoLVM (those nodes have no storage and the per-node classes won't bind there).

```
spec:
  nodeSelector:
    node.antbrains.com/role: invoker          # roso + poco
  tolerations:
    - key: dedicated                          # needed to land on poco (tainted);
      operator: Equal                         # harmless on roso (untainted)
      value: invoker
      effect: NoSchedule
  # Leave priorityClassName unset (default/normal). Do NOT set system-*-critical:
  # on roso the control-plane pods are system-critical and must be able to
  # preempt invoker pods under pressure (see §6.1). Set resource requests/limits
  # on invoker pods so the scheduler honours roso's reservations.
  # priorityClassName: "" # normal
```

Prefer poco for heavy invoker load. `poco` (630Gi, 80c) has no control plane, so put the bulk of invoker workloads there; keep `roso`'s share light so etcd/apiserver always have headroom. `roso` is untainted and `poco` is tainted, so a pod with the spec above can land on either — add a stronger `nodeAffinity` toward `poco` if you want to bias scheduling.

11. Rancher (optional management UI)

Rancher is exposed via NPM (proxy host → `rancher.cattle-system.svc`), TLS in NPM.

```
helm repo add rancher-stable https://releases.rancher.com/server-charts/stable
helm repo update
kubectl create namespace cattle-system
helm install rancher rancher-stable/rancher -n cattle-system \
  --set hostname=rancher-uslab.eng.antbrains.com \
  --set bootstrapPassword=admin \
  --set ingress.enabled=false \
  --set tls=external
kubectl -n cattle-system rollout status deploy/rancher
```

Then add the NPM Proxy Host. **The forward MUST be HTTPS:443, not HTTP:80** — Rancher 302-redirects HTTP→HTTPS, and proxying HTTPS to its plaintext :80 throws `SSL ... wrong version number` → `502` (we hit both). Working config:

NPM Proxy Host field	Value
Domain	<code>rancher-uslab.eng.antbrains.com</code>
Scheme	<code>https</code>
Forward Hostname	<code>rancher.cattle-system.svc.cluster.local</code>
Forward Port	<code>443</code>
Websockets Support	ON (Rancher UI needs it for live logs / shell)
SSL	request a cert, Force SSL , HTTP/2

Reach it over `https://` (not http). On first load Rancher sets its `server-url` ; set it explicitly if you want it deterministic:

```
kubectl patch settings.management.cattle.io server-url --type merge \
-p '{"value":"https://rancher-uslab.eng.antbrains.com"}'
```

 **Lock this down.** Rancher is full cluster-admin. If its hostname resolves to a public IP (ours does), restrict the NPM Proxy Host with an **Access List** (IP allowlist / basic-auth) or keep it LAN/VPN-only. See [ACCESS-STRATEGY.md](#) .

12. Observability (optional first pass)

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm install kps prometheus-community/kube-prometheus-stack \
-n monitoring --create-namespace
```

Scrape: TopoLVM (controller/lvmd metrics), VolSync (`volsync-system`), MetalLB (controller/speaker), node-exporter LVM/VG free space. Thick = alert on **VG free space** (not thin-pool Data%/Meta%).

13. Build Order

1. **\$0** — fix VIP, MetalLB pool, IPs, domain, passwords.
2. **\$4** on all 6 nodes. **\$4.1** disk + VG (thick) on the 4 app nodes only.
3. **\$5** — HAProxy + keepalived (API VIP) on `roso` .
4. **\$6** — RKE2 server on `roso` .
5. **\$7.1** join app workers · **\$7.2** join `poco` · **\$7.3** verify · **\$7.4** cert-manager.
6. **\$8.1–8.4** — label, install TopoLVM, StorageClasses, smoke-test.
7. **\$8.5** — VolSync (optional).
8. **\$9.1** — MetalLB · **\$9.2** — NPM · **\$9.3** verify.
9. **\$11** — Rancher (optional) · **\$12** — observability (optional).

Phase 2 (a few months out): migrate the edge to Traefik v3 + cert-manager (`Traefik.phase2/`); optionally switch storage to thin pools if zero-downtime snapshot replication becomes a hard requirement.

14. Health Checks

```
# Nodes & scheduling
kubectl get nodes -o wide
kubectl get pods -A | grep -vE 'Running|Completed|^NAMESPACE' # should be empty

# TopoLVM
kubectl -n topolvm-system get pod -o wide
kubectl get csidrivers | grep topolvm.io
kubectl get csistoragecapacities -A -o wide
kubectl get sc # no default class

# On each storage node - VG free space (thick: simple)
ssh cheese 'sudo vgs vg-topolvm-cheese'
ssh dough 'sudo vgs vg-topolvm-dough'
ssh nicks 'sudo vgs vg-topolvm-nicks'
ssh rosatini 'sudo vgs vg-topolvm-rosatini'

# VolSync
kubectl -n volsync-system get pod
kubectl get replicationsource,replicationdestination -A

# MetalLB + NPM
kubectl -n metallb-system get pod,ipaddresspool,l2advertisement
kubectl -n npm get pod,svc -o wide
kubectl get svc -A | grep LoadBalancer

# kube-API LB
ip -br addr | grep 10.35.0.60 # on roso
curl -fsS http://10.35.0.60:8404/stats -u admin:<STATS_PW> -o /dev/null && echo OK
```

15. Reference & migration notes

- **Why TopoLVM over Ceph here?** Raw NVMe latency — Ceph routed every I/O over the network through OSD/RADOS daemons; even at [size:1](#) that hop dominated. TopoLVM keeps I/O on-node (~10–40× lower latency). See [storage-performance.html](#).
- **Why thick over thin?** Lowest, flattest latency, nothing to monitor, and it matches the Lumens cluster. Cost: no CSI snapshots → VolSync runs Direct. Comparison: [thin-vs-thick.html](#), [thin-vs-thick-implementation.html](#).
- **Why no ingress yet?** Initial stage favours simplicity: MetalLB + NPM cover edge + TLS via a UI. Traefik v3 + cert-manager are parked in [Traefik.phase2/](#).
- **Old uslab cluster** (10.48/10.49) used [node-ip / advertise-address](#) to pin RKE2 to the right NIC — reused in §6.
- **CIDR layout** (10.50 / 10.51) deliberately does not overlap old punelab or uslab clusters, so the three can peer if needed.
- **Access strategy** for developers/users is a separate doc: [ACCESS-STRATEGY.md](#).

16. As-built status & gotchas hit during bring-up (2026-06-19)

What's deployed and the issues we resolved — read this before touching the cluster.

Deployed & healthy

- 4 nodes joined: [roso](#) (control-plane), [cheese](#), [dough](#), [rosatini](#). [nicks](#) and [poco](#) **not yet joined** (fill [nicks](#) / [poco](#) config + [<IP_poco>](#)).
- TopoLVM thick, per-node VGs created on [cheese](#) / [dough](#) / [rosatini](#) (~3.73 TiB each), classes [local-ssd-*](#) applied, capacity reported via CSIStorageCapacity.
- MetalLB pool [10.35.0.61–10.35.0.99](#); **NPM** Running on [cheese](#), LB IP [10.35.0.61](#). **Rancher** Running (3 replicas), reachable through NPM at [https://rancher-uslab.eng.antbrains.com/](#).

Four gotchas we hit (now baked into the plan)

1. **roso taint deadlocked the CNI bootstrap.** [dedicated=invoker:NoSchedule](#) on the only control-plane node left [helm-install-rke2-calico/-coredns](#) Jobs Pending → workers NotReady. **Fix:** [roso untainted](#) (§6/§6.1).
2. **TopoLVM Insufficient topolvm.io/capacity.** The pod mutating webhook injected an extended-resource request no node advertised (no scheduler extender). **Fix:** [webhook.podMutatingWebhook.enabled: false](#) (§8.2).
3. **NVMe Device or resource busy.** Disks shipped with an old [ubuntu-vg](#) LVM layout. **Fix:** [tear down old LVM/partitions first](#) (§4.1).
4. **Rancher 502 behind NPM.** Forwarding to HTTP:80 (Rancher redirects to HTTPS) / HTTPS→:80 (wrong version). **Fix:** [NPM forward = HTTPS:443 + websockets](#) (§11).

17. NPM availability & HA

Current: NPM is [replicas: 1](#), [Recreate](#), pinned to [cheese](#) by its RWO node-local PVCs ([local-ssd-cheese](#)). It survives **process/pod crashes** (kubelet restarts it) but **NOT a cheese node failure** — the RWO PVC is stuck on the dead node, so the pod can't reschedule. NPM is the single

front door for all apps, so treat this as a known SPOF.

NPM can't be made cleanly HA (single SQLite DB + local cert files; not built for active-active). Options, worst→best for HA:

Option	Survives node failure	Effort	Notes
1. Single + off-node backup	❌ (fast manual restore)	Low	Add a <code>PodDisruptionBudget</code> ; back up <code>/data</code> + <code>/etc/letsencrypt</code> off-node (VolSync Direct or a CronJob → PVC on <code>dough</code>); keep a restore runbook. Removes data-loss risk, not downtime.
2. Replicated storage (Longhorn)	✅ auto	Medium	NPM stays single-replica but its volume is mirrored across nodes; pod reschedules + reattaches on another node if <code>cheese</code> dies. Adds a 2nd storage system.
3. Traefik v3 + cert-manager 🌟	✅ true HA	Medium	Stateless ingress: config from API, certs in Secrets (etcd). Run 2–3 replicas (anti-affinity) behind MetalLB — any node can die. No node-local PVC. This is the Phase-2 edge (<code>Traefik.phase2/</code>).

Recommendation: for the initial stage do **Option 1** (cheap, kills data-loss risk). For genuine HA edge, bring **Phase 2 (Option 3)** forward — don't try to bolt HA onto NPM.